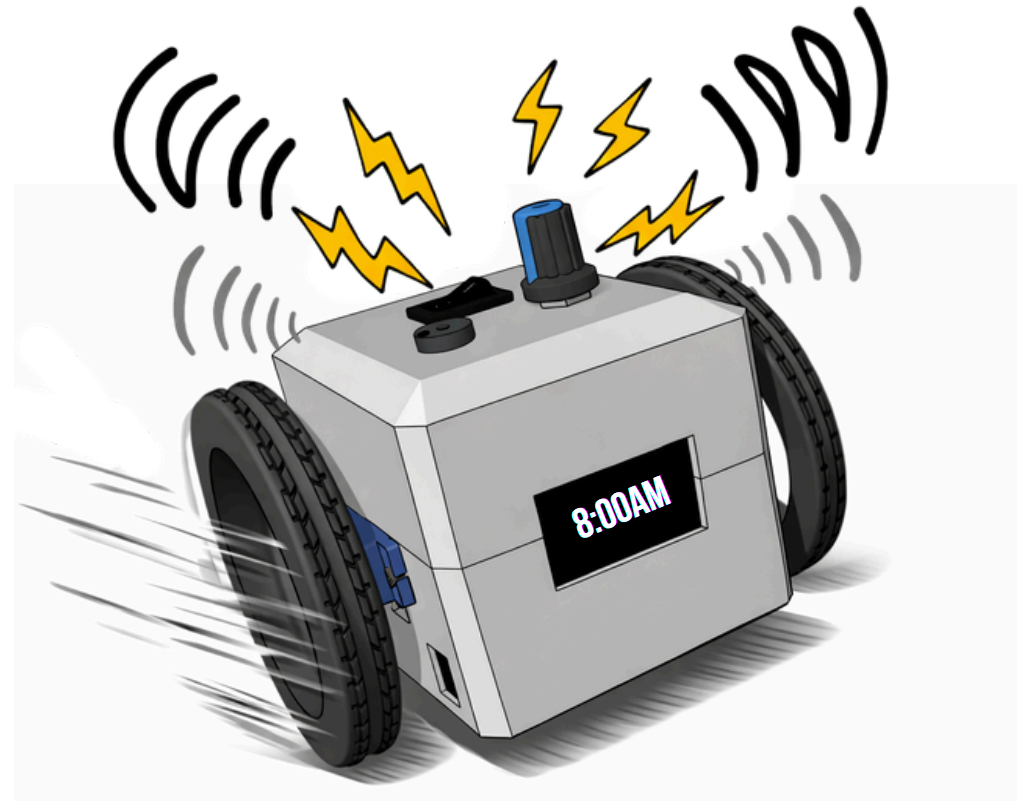
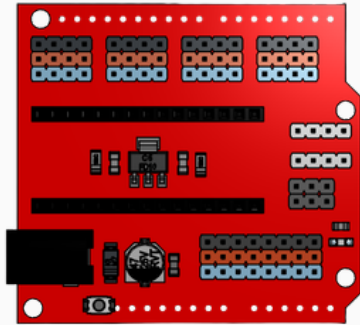


ALARM BOT

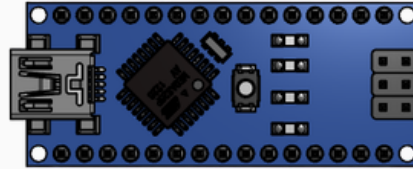
BUILDING & PROGRAMMING MANUAL



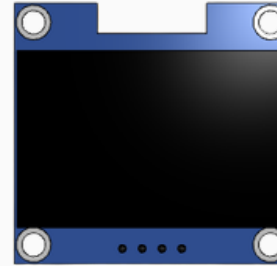
WHAT'S IN THE KIT?



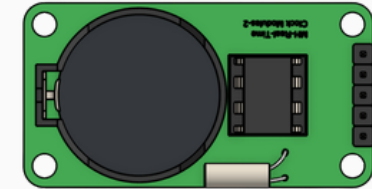
EXPANSION SHIELD
(x1)



ARDUINO NANO
(x1)



SCREEN
(x1)



CLOCK MODULE
(x1)



COIN BATTERY
(x1)



JUMPER WIRES
(x20)



KNOB
(x1)



KNOB NUT
(x1)



KNOB CAP
(x1)



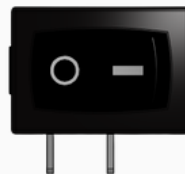
BATTERY HOLDER
(x1)



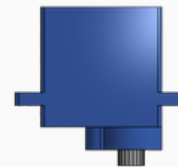
BUZZER
(x1)



SCREWDRIVER
(x1)



SWITCH
(x1)



MOTOR
(x2)

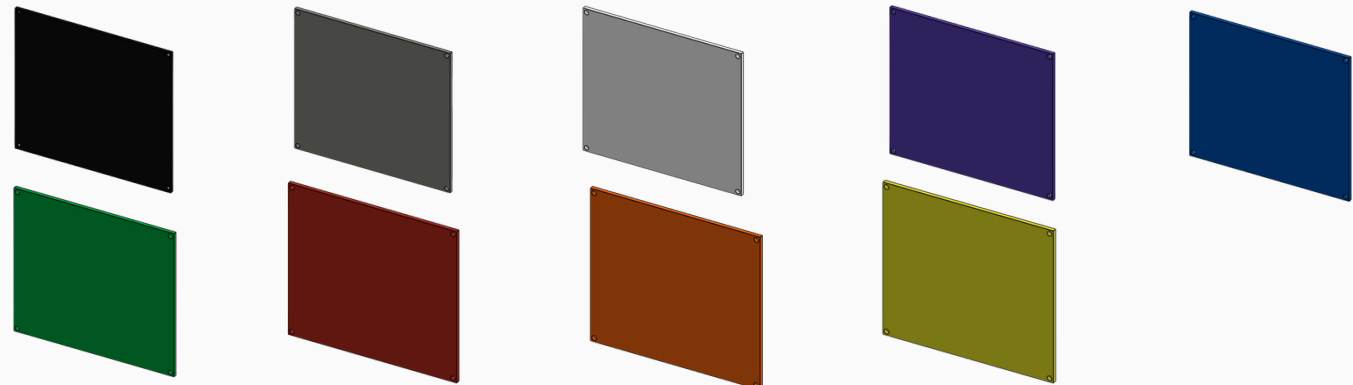
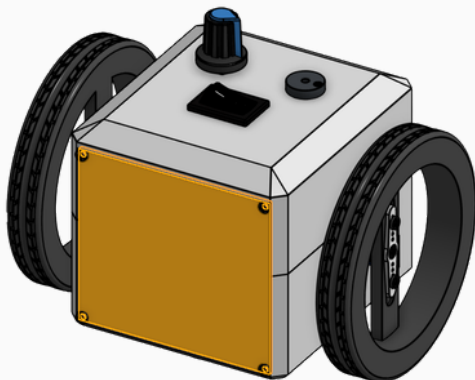
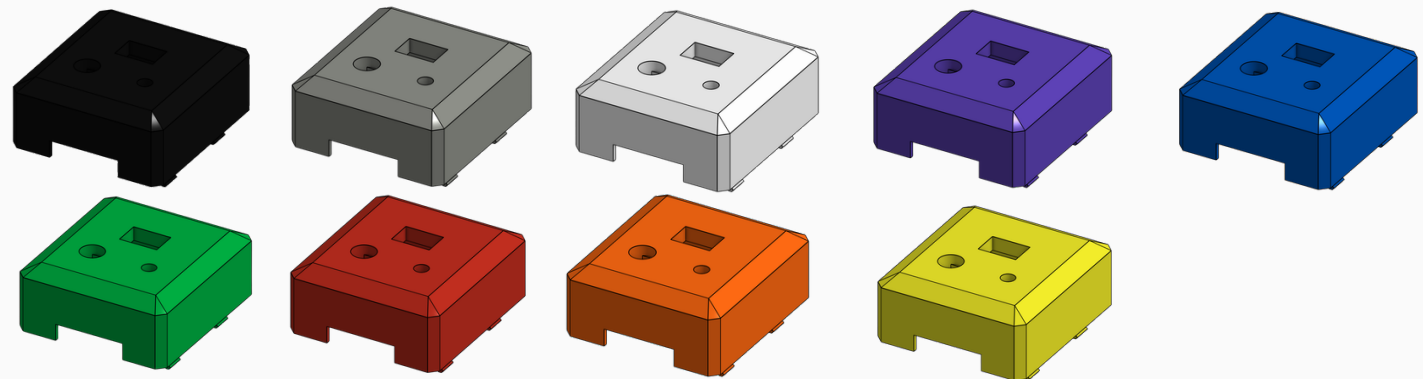
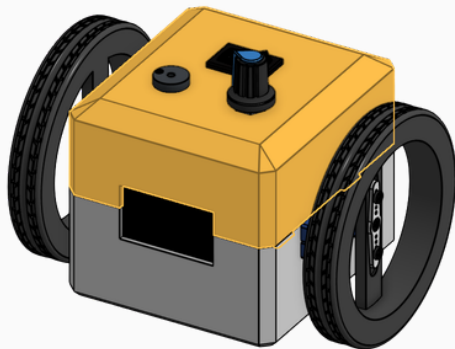
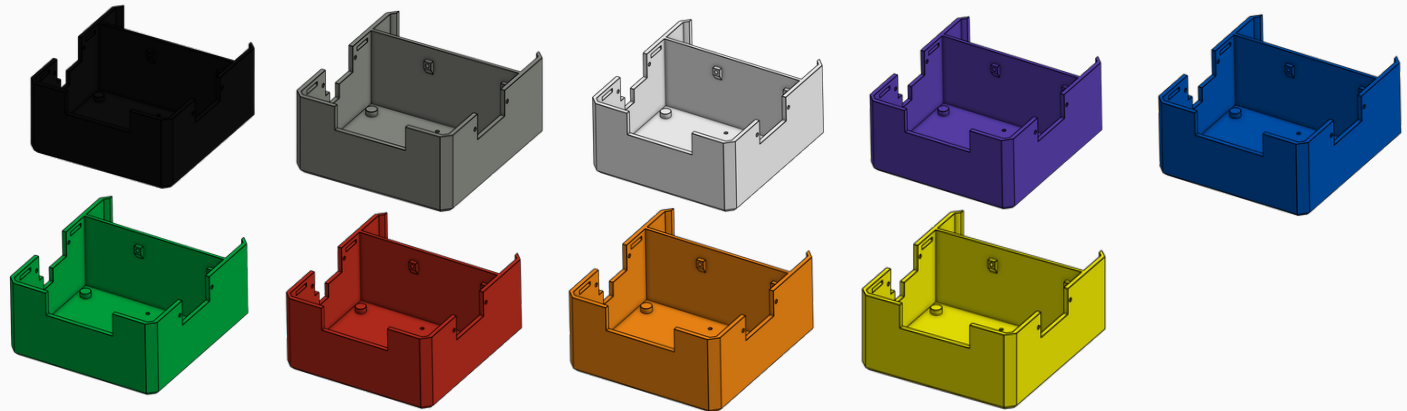
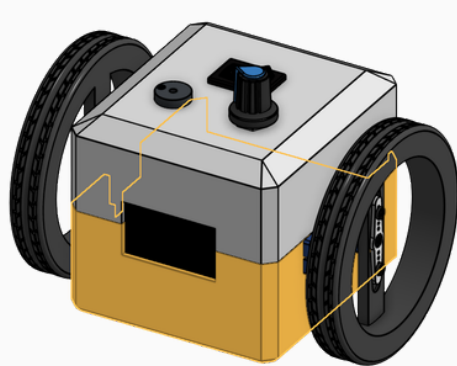


SHORT SCEWS
(x9)



LONG SCREWS
(x6)

PICK YOUR COLOURS!



BEFORE BUILDING

If you need help, don't be afraid to ask!

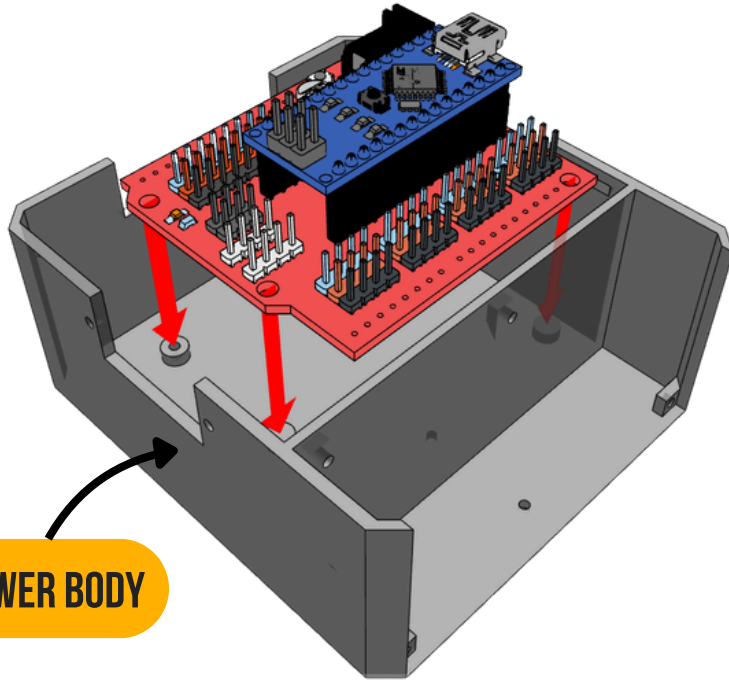
When screwing, look for these icons indicating which screws to use.



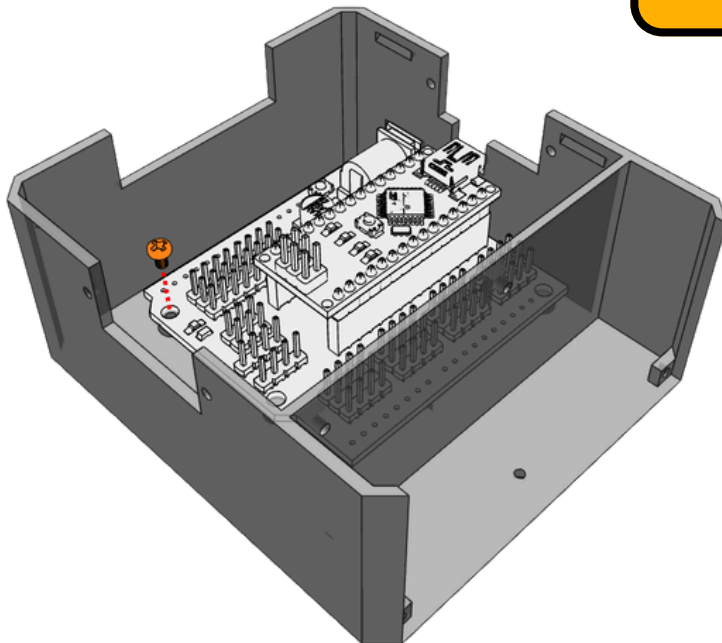
After every **CHECKPOINT** please ask one of our helpers to check on your progress

Please keep volume down. Though we encourage a playful environment, we ask that you keep discussions amongst your neighbours only.

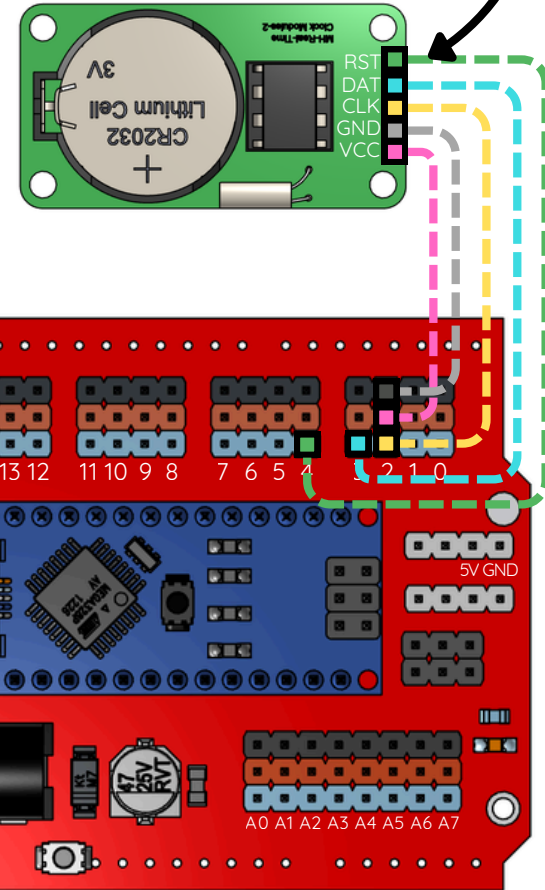
1



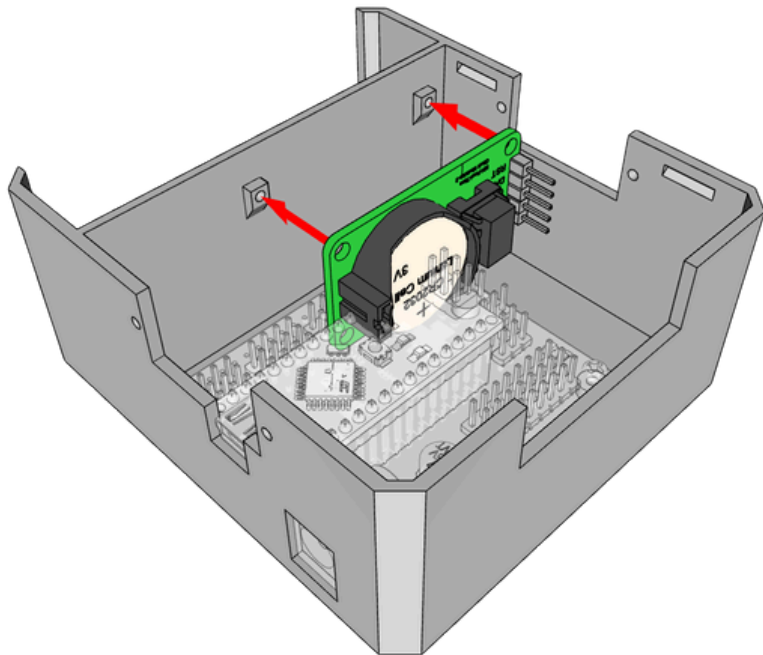
2



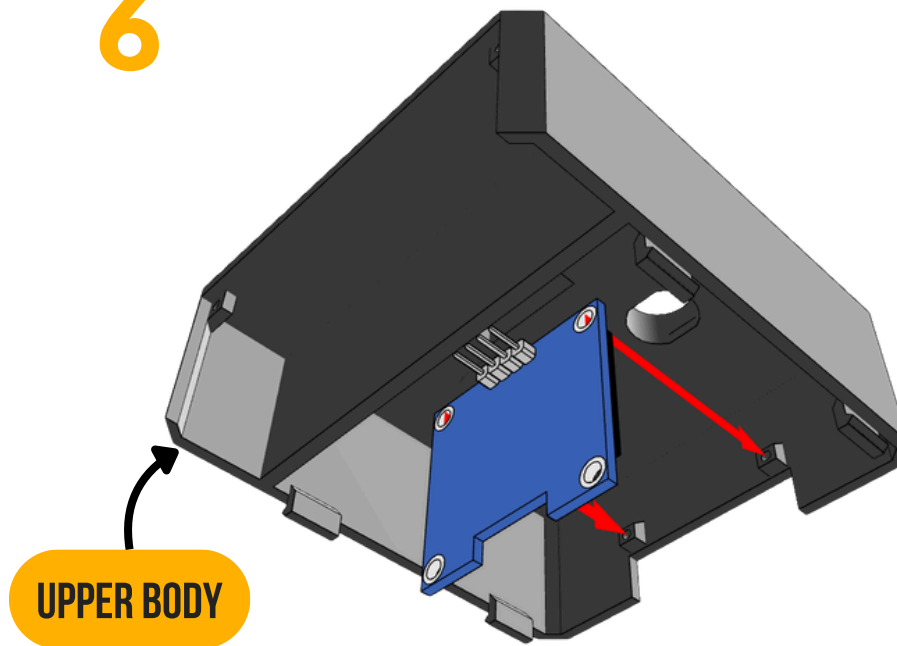
3
CHECKPOINT



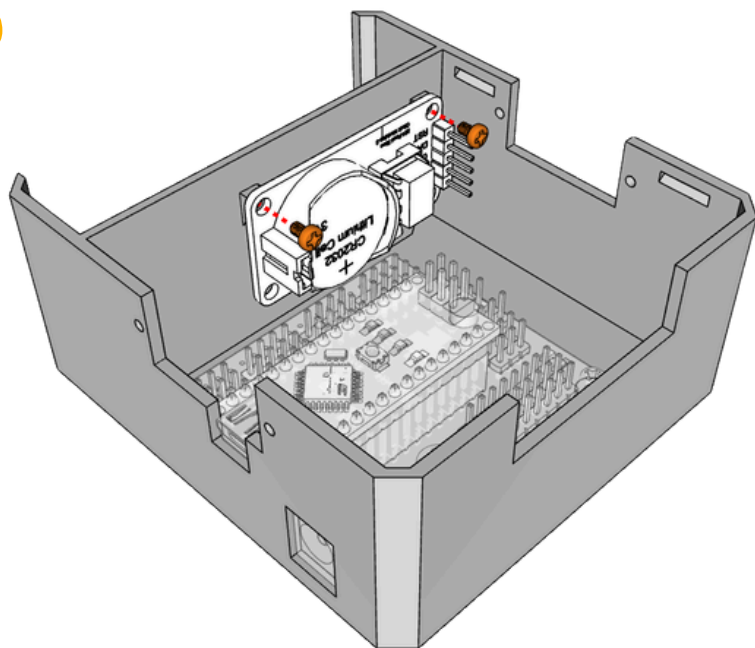
4



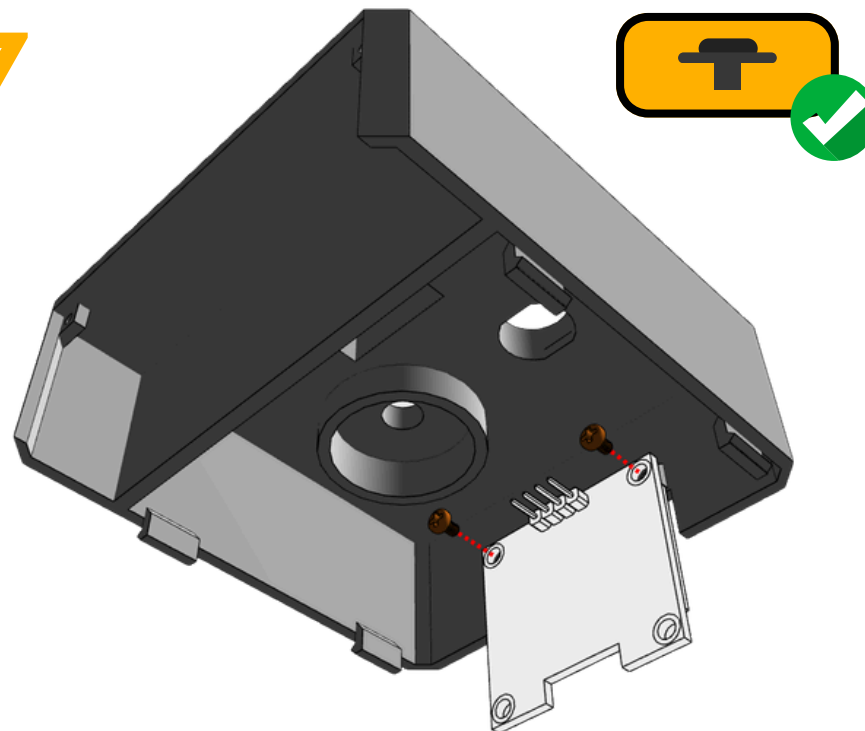
6



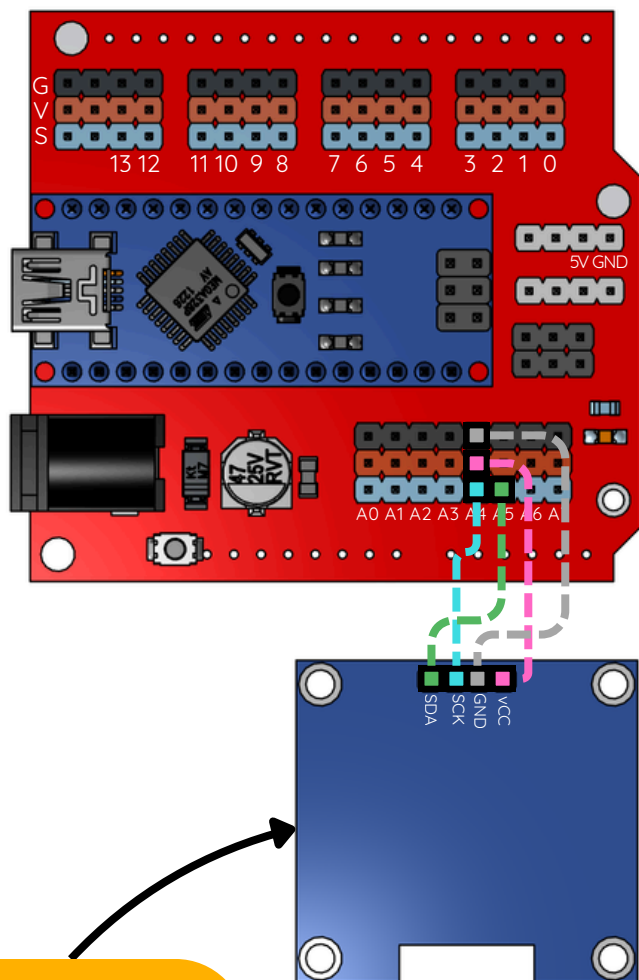
5



7

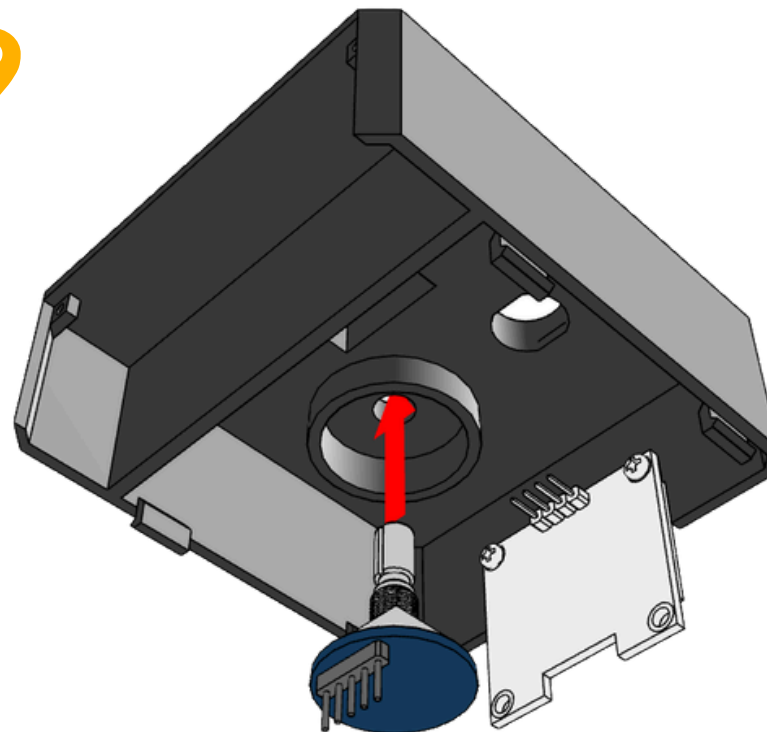


8

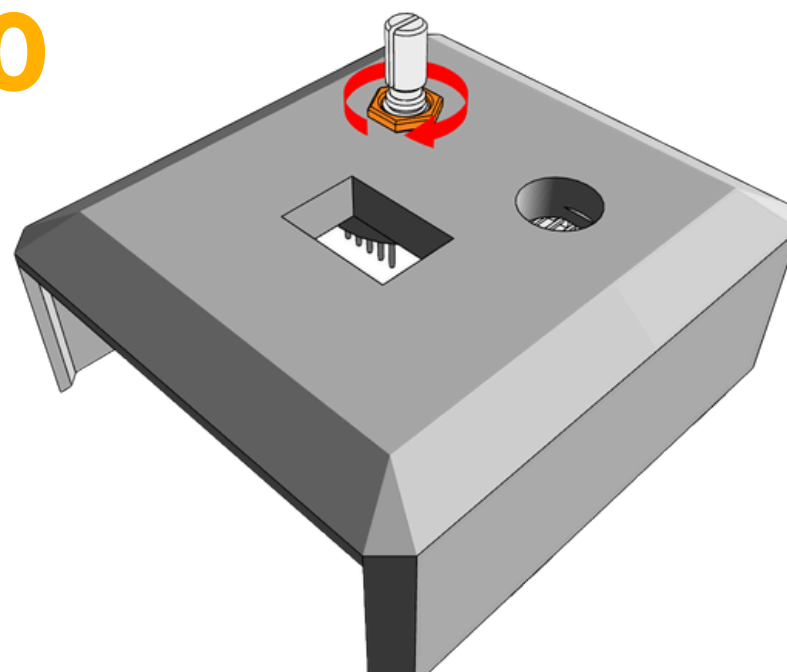


**BACK VIEW
OF THE SCREEN**

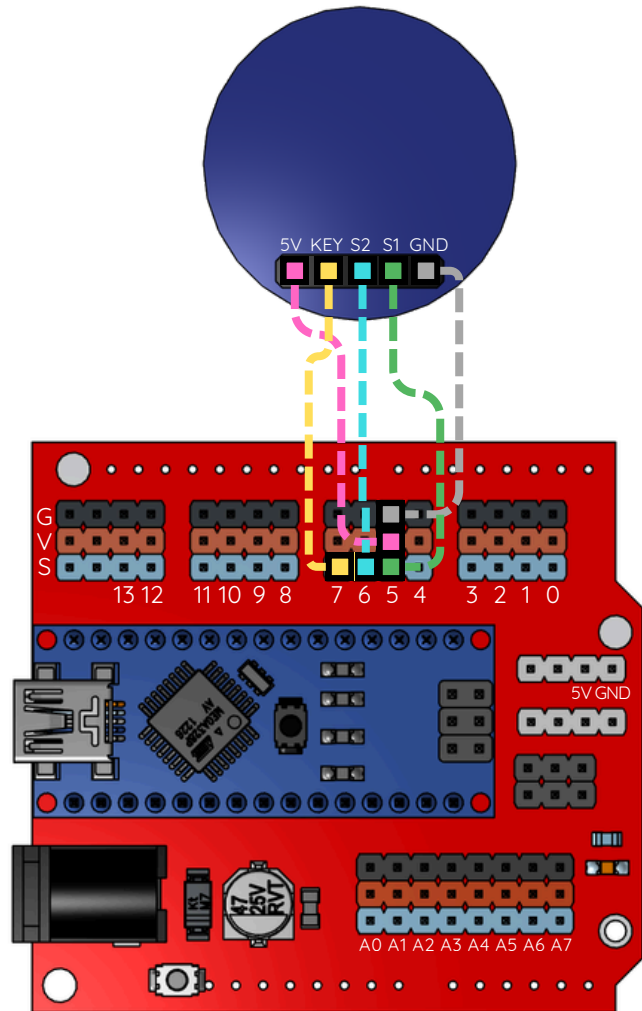
9



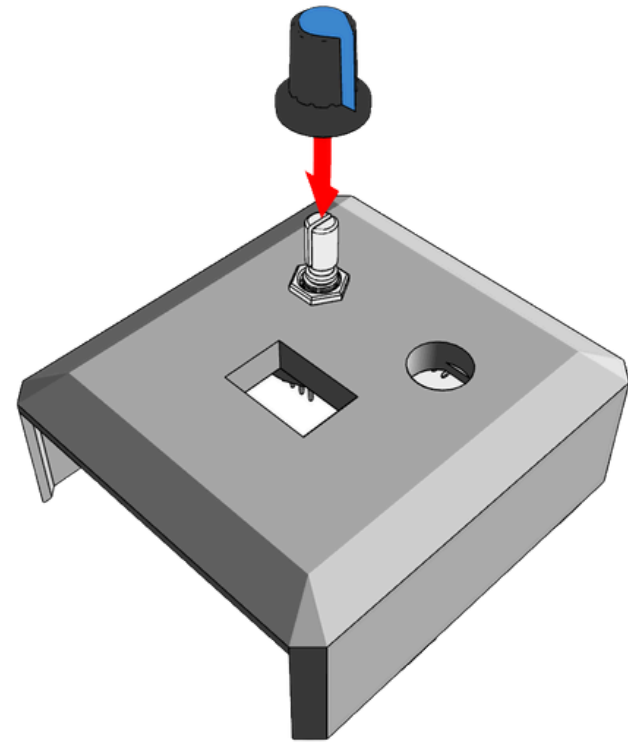
10



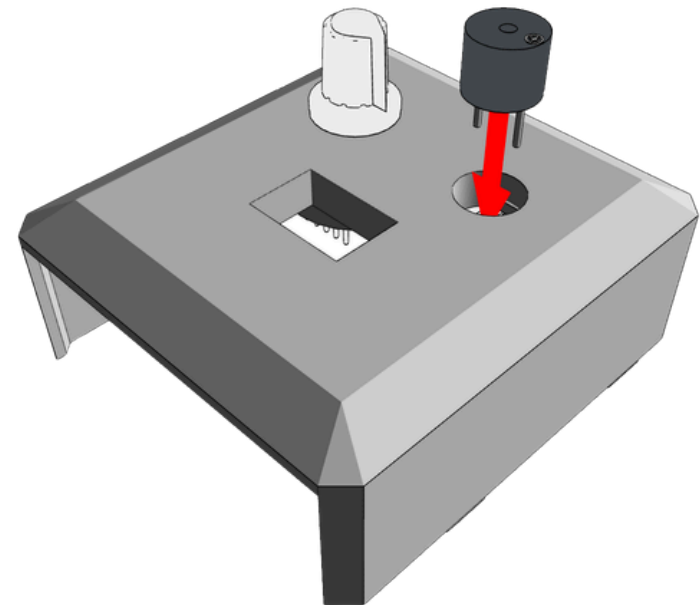
11



12

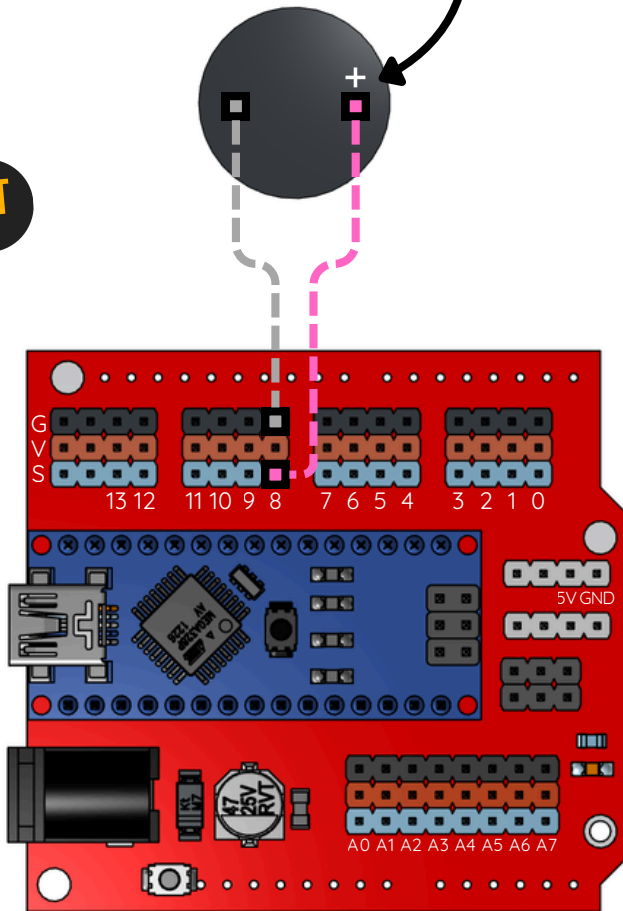


13



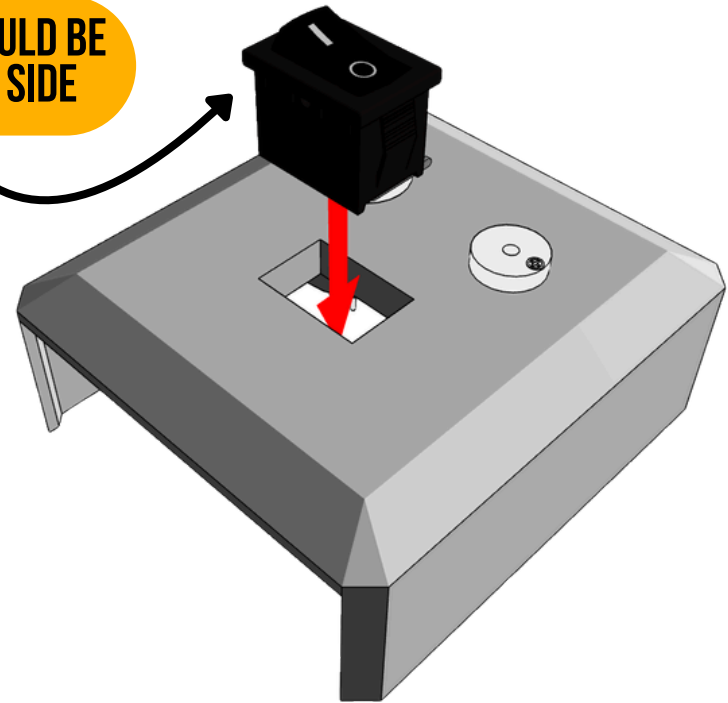
THE "+", CAN BE FOUND ON
TOP OF THE BUZZER

14
CHECKPOINT



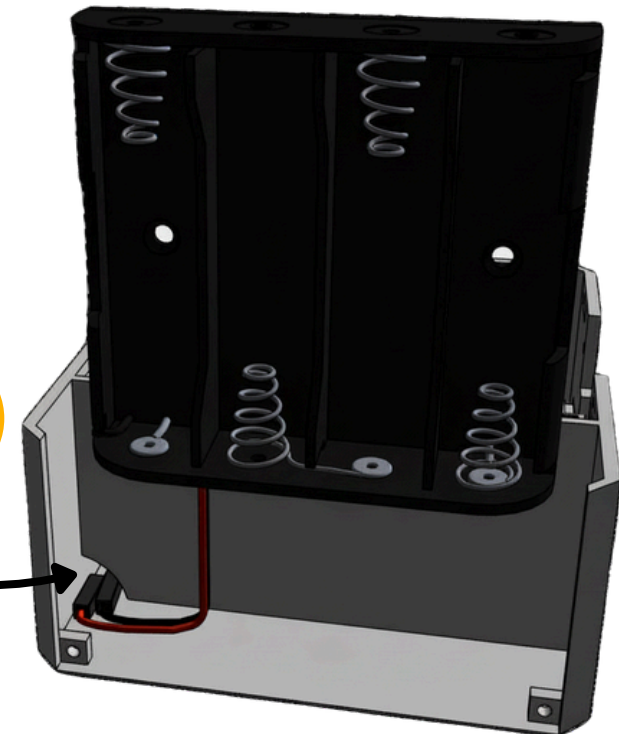
THE "I", SHOULD BE
FACING THIS SIDE

15

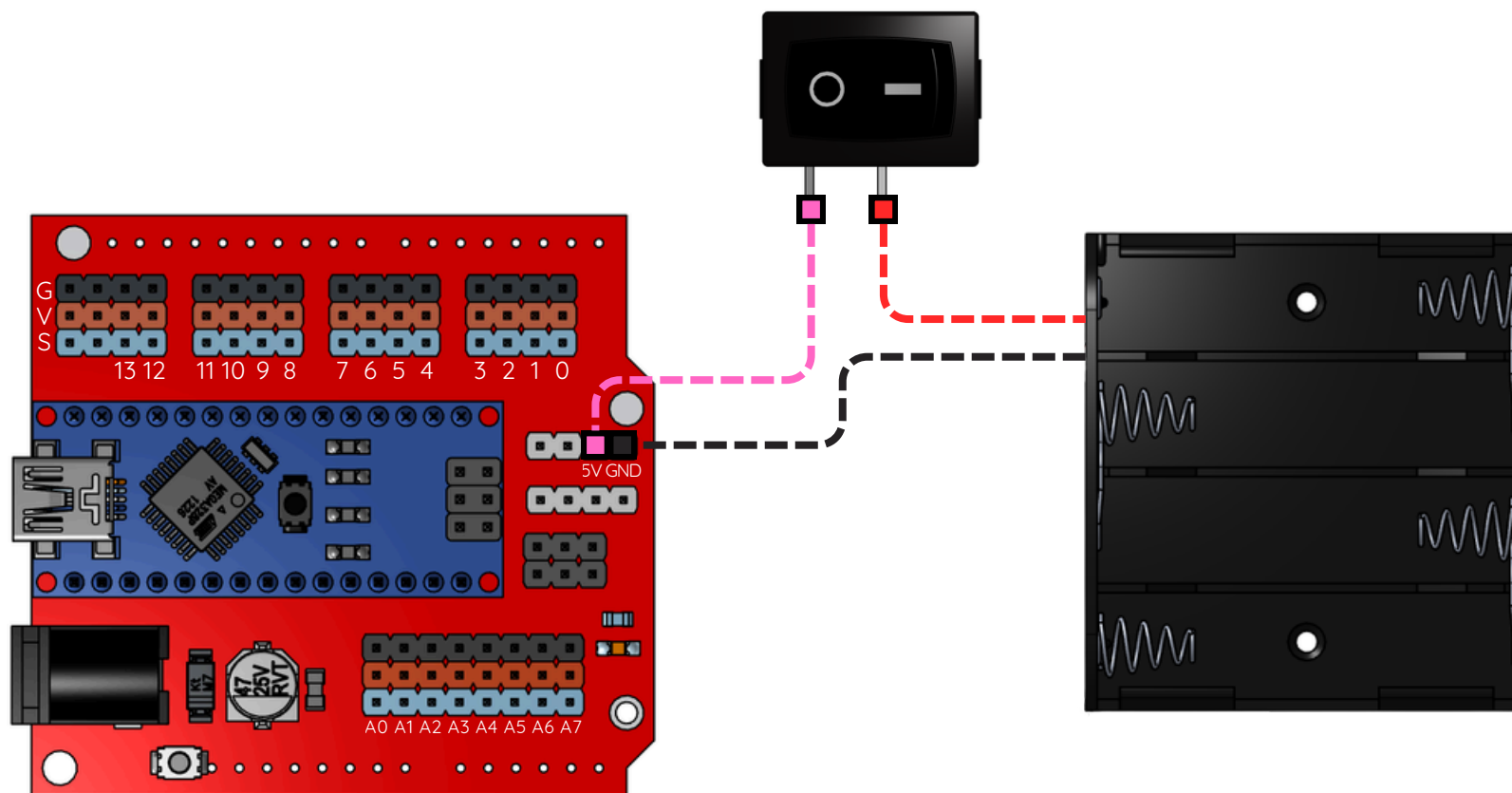


16

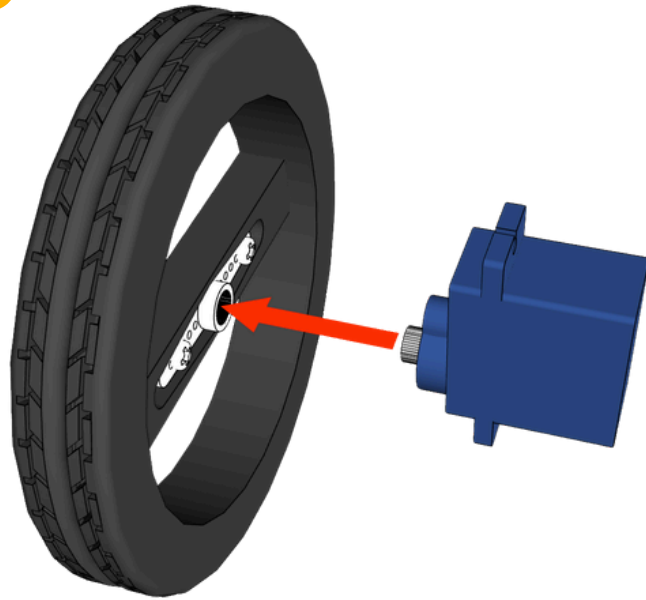
PUSH THE WIRES
THROUGH THIS HOLE



17 CHECKPOINT



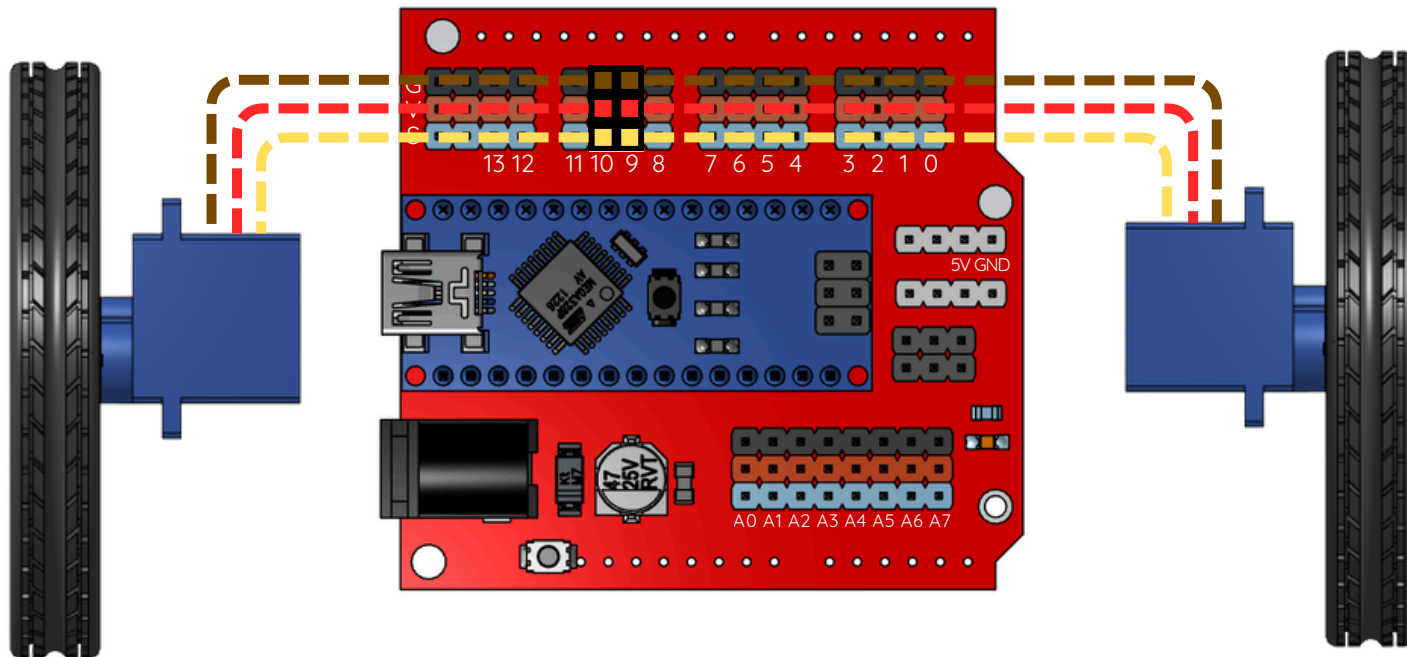
18



19

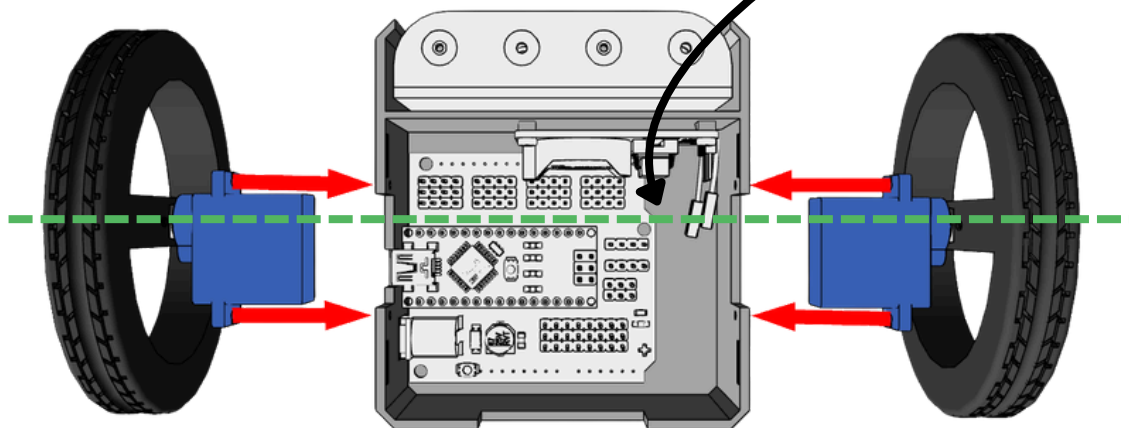


20

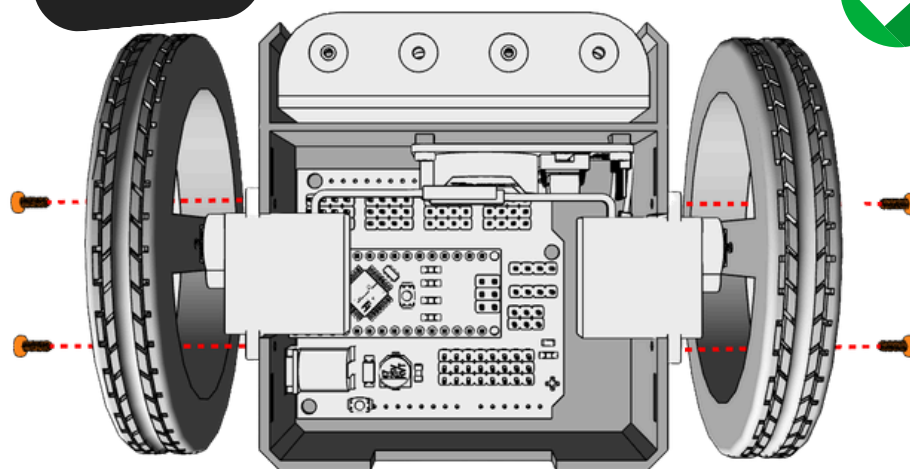


21

ALIGN THE WHEELS
ALONG THIS LINE

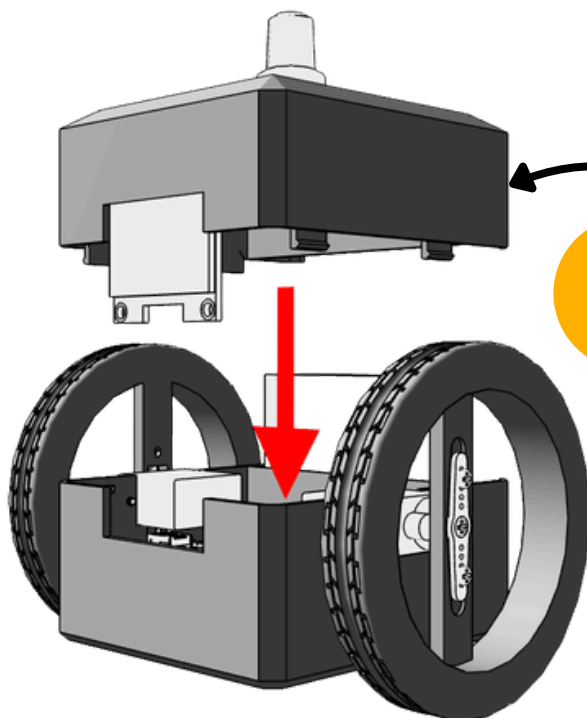


22
CHECKPOINT

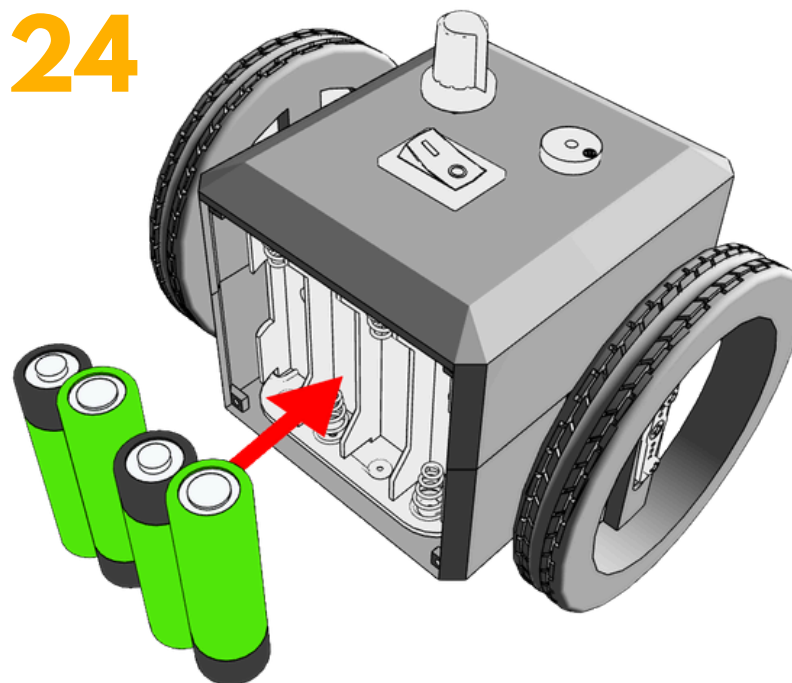


23

SQUEEZE THE SIDES
FOR AN EASIER FIT



24

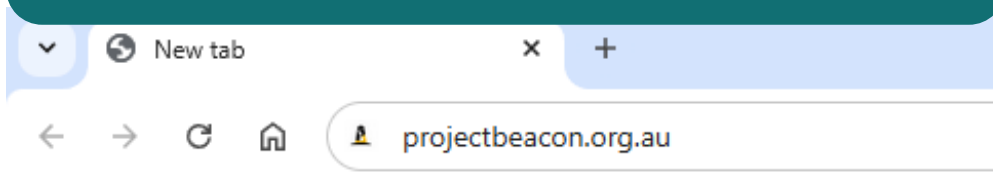


PROGRAMMING TIME!

Congratulations on reaching the programming section. In this section we will teach you how to code the bot's alarm tone, it's movement path and finally how to integrate these codes into the final overall code.

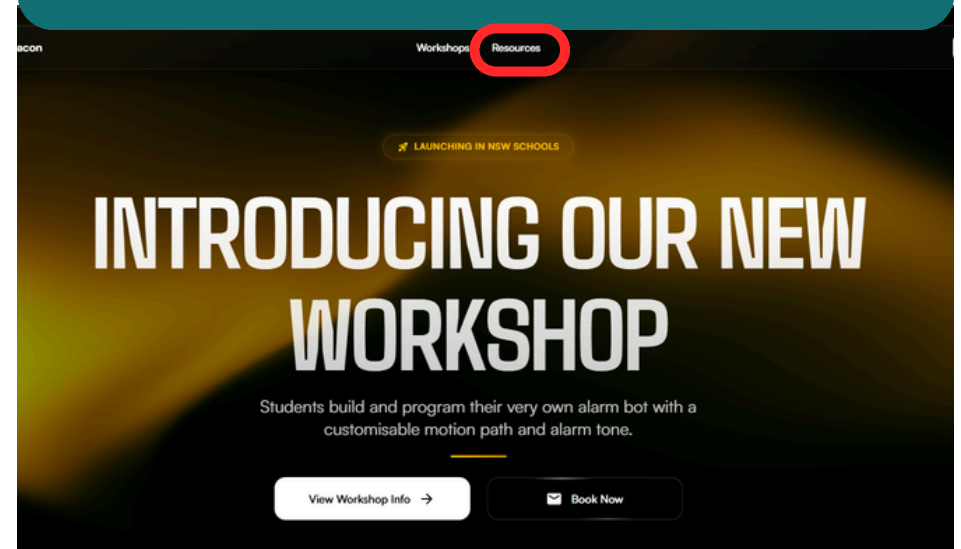
1

Access our website at projectbeacon.org.au



2

Click on the resources tab at the top of the page.



3

Enter the access code provided by your instructor

Resources



Protected Resources

These resources are available to registered schools. Please enter your access code.

ACCESS CODE

Enter access code

Unlock Resources

4

Under Workshop Code Templates select Alarm Bot Workshop

Resources

Comprehensive materials for educators. Access student manuals, code templates, and curriculum documentation.

Student Resources



Student Build Manual

Step-by-step assembly guide with diagrams and safety instructions.

2.4 MB



Workshop Code Templates

Access pre-written Arduino code for each workshop project.



Alarm Bot Workshop

5

Press "Download" for all three code templates. You should have three files named movement-code.ino, alarm-code.ino, and overall-code.ino

Code Templates

Copy and paste ready-to-use templates for your projects.

Movement Code

Basic Movement Code Template



```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
#include <Servo.h>
```

```
void beginMovementSequence()
void endMovementSequence()
void moveLeftMotorCm(int speed);
void moveRightMotorCm(int speed);
void moveWaitCm(unsigned long durationMs);
```

```
#define leftMotor() moveLeftMotorCm()
```

Overall Code

Overall Code Template



Alarm Code

Basic Alarm Code Template



```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
```

```
void beginSoundSequence()
void endSoundSequence()
void soundWaitCm(unsigned long durationMs);
void soundPlayToneCm(int pitch, unsigned long durationMs);
```

```
#define playTone(pitch, sec) soundPlayToneCm(pitch, (unsigned long)(sec * 1000))
```

```
#define wait(sec) soundWaitCm(unsigned long)(sec * 1000)
```

```
void studentAlarmSound() {
```

6

Press the "Downloads" button on the top right of your browser and open alarm-code.ino. (If you are asked what app to open the file with, select Arduino IDE)



Recent download history



overall-code.ino

6.0 KB • Done



alarm-code.ino

4.1 KB • Done



movement-code.ino

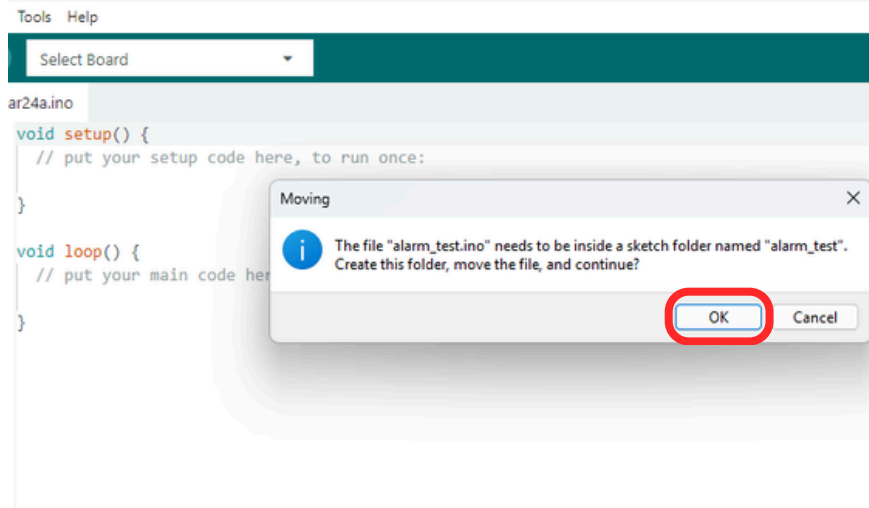
4.1 KB • Done

Full download history



7

Press "OK" when asked to create a new folder



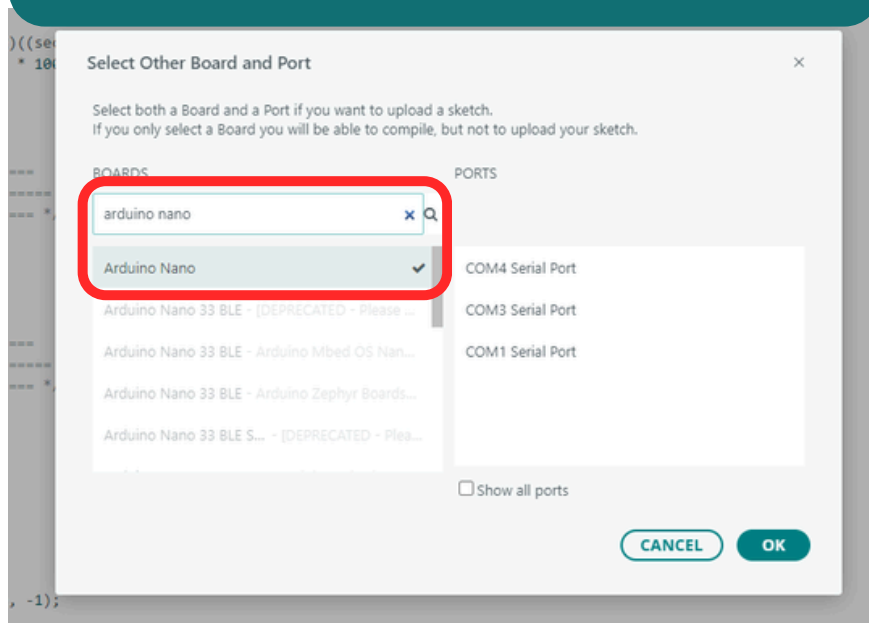
8

Click "Select Board" on the top-left of your screen then "Select other board and port"



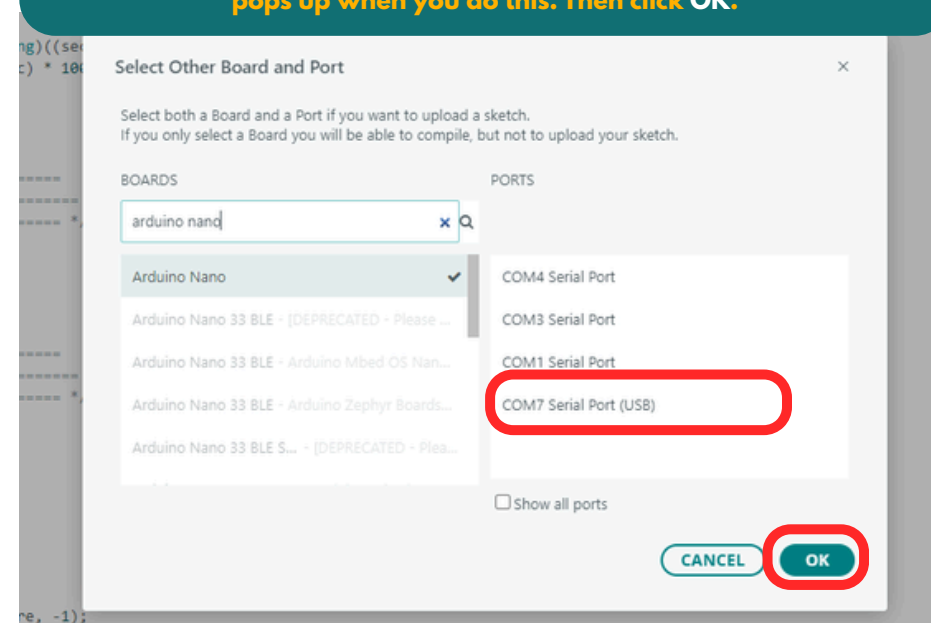
9

In the search bar type "Arduino Nano", and click on it.



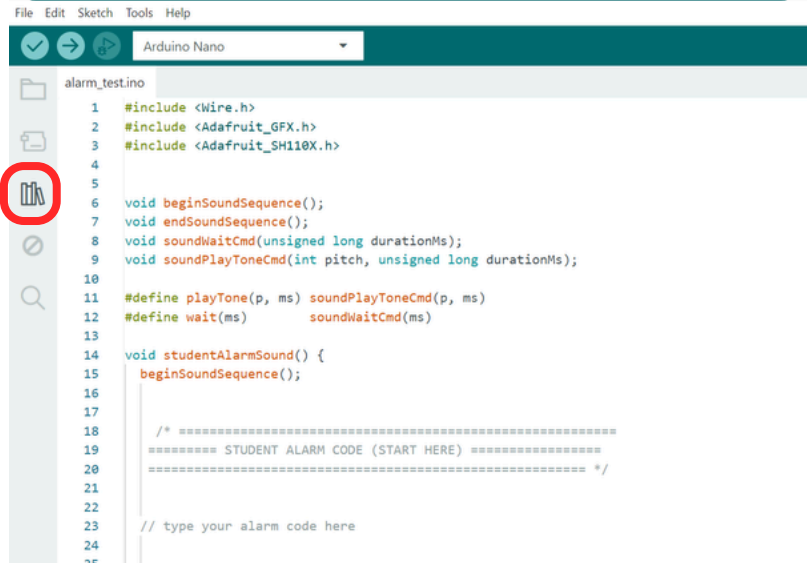
10

Connect your bot to a USB port with the provided blue cable. Select the new COM Serial Port that pops up when you do this. Then click OK.



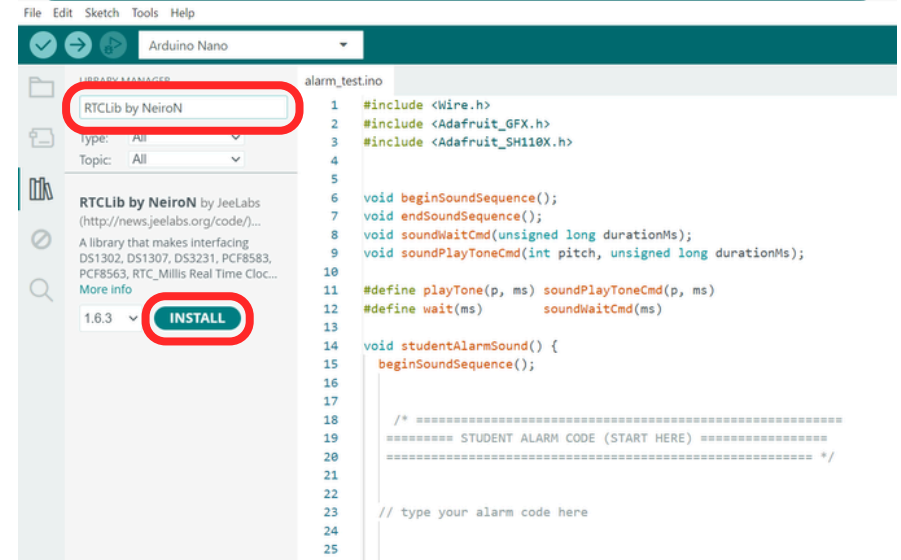
11

Click on the bookshelf on the side to open the Library Manager



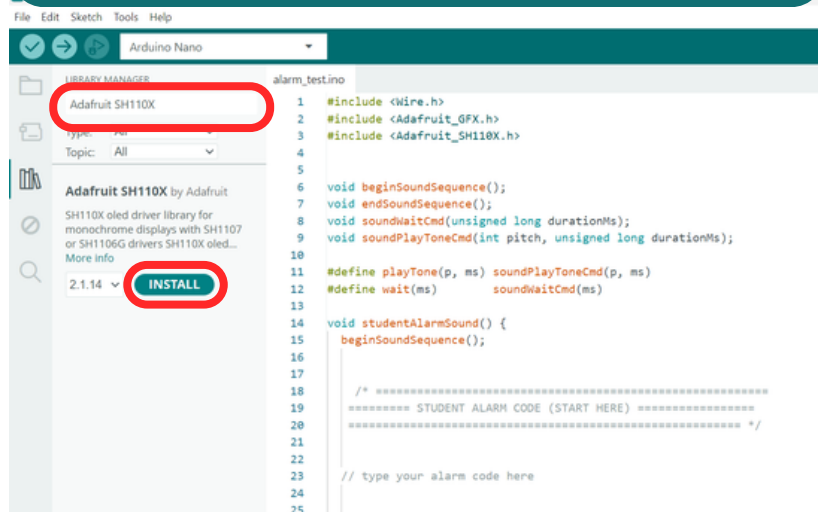
12

Search "RTCLib by NeuroN" and click INSTALL



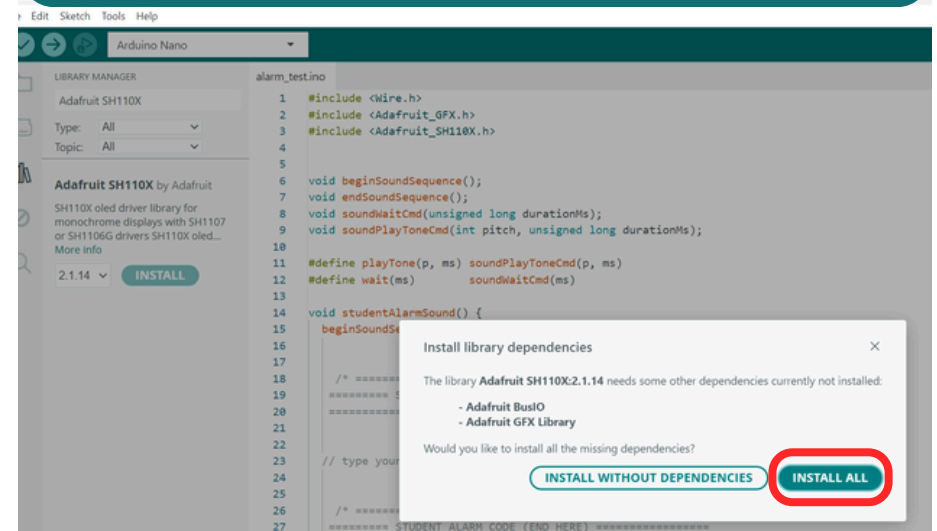
13

Search "Adafruit SH110X" and click INSTALL



14

When asked to install library dependencies select "INSTALL ALL"



15

Type your alarm code into the STUDENT ALARM CODE section

```

16 // type your alarm code here
17 // type your alarm code here
18 /* ===== STUDENT ALARM CODE (START HERE) ===== */
19 ===== STUDENT ALARM CODE (START HERE) =====
20 ===== */
21 // type your alarm code here
22 // type your alarm code here
23 // type your alarm code here
24 // type your alarm code here
25 // type your alarm code here
26 /* ===== STUDENT ALARM CODE (END HERE) ===== */
27 ===== STUDENT ALARM CODE (END HERE) =====
28 ===== */
29 endSoundSequence();
30 }
31 #undef playTone

```

Delete this line and replace with playTone and wait commands

playTone(pitch, duration);

High pitch note → 2000
Low pitch note → 300

Duration of the note
(in seconds)

Examples:

- playTone(1800, 5); → High pitched note that runs for 5 seconds
- playTone(800, 15); → Moderately low pitched note that runs for half a second

wait(duration);

Duration of silence from
one note to the next note

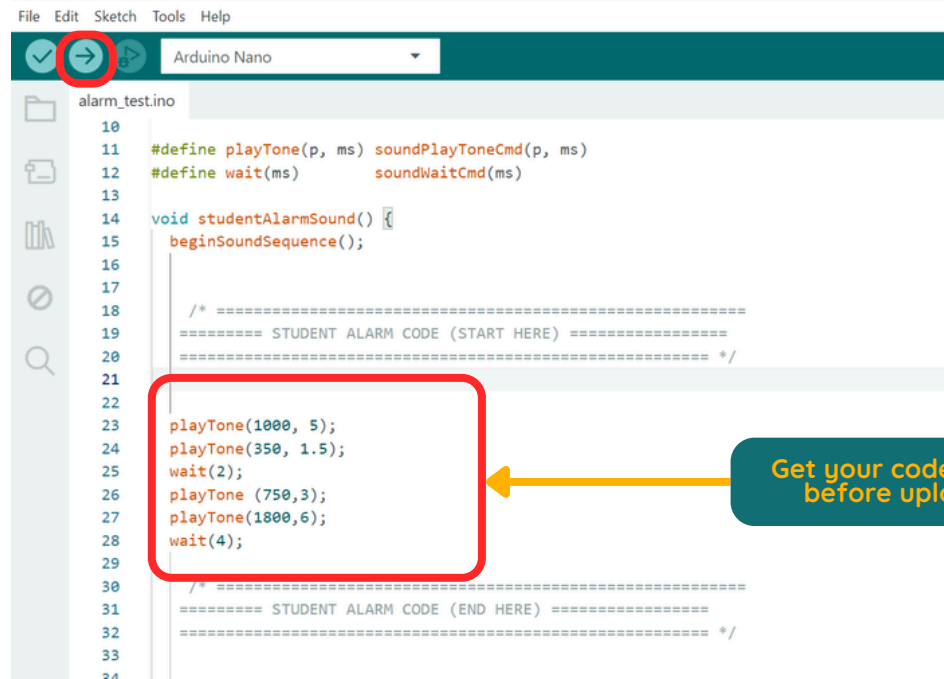
Examples:

- playTone(400, 10);
wait(2); → 2 second silence before the next playTone command

You can find some example code of famous tunes on our website!

16

Click on the top-left arrow to upload the code



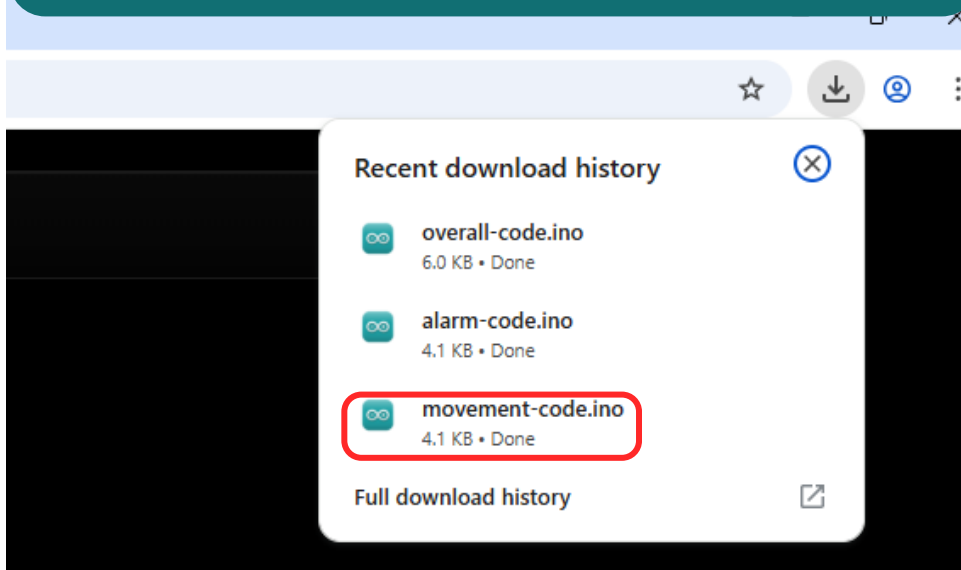
The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. Below the menu bar is a toolbar with icons for checking, uploading, and compiling. The upload icon (a right-pointing arrow) is circled in red. To the right of the toolbar is a dropdown menu showing 'Arduino Nano'. The main workspace displays a C++ code file named 'alarm_test.ino'. The code includes function definitions for 'playTone' and 'wait', and a 'void studentAlarmSound()' function. Inside 'studentAlarmSound()', there is a call to 'beginSoundSequence()' followed by a large block of commented-out code. A red box highlights a section of code within this block: 'playTone(1000, 5);', 'playTone(350, 1.5);', 'wait(2);', 'playTone(750, 3);', 'playTone(1800, 6);', and 'wait(4);'. A yellow arrow points from a callout box to this highlighted code.

```
10
11 #define playTone(p, ms) soundPlayToneCmd(p, ms)
12 #define wait(ms) soundWaitCmd(ms)
13
14 void studentAlarmSound() {
15     beginSoundSequence();
16
17
18     /* =====
19     ===== STUDENT ALARM CODE (START HERE) =====
20     ===== */
21
22     playTone(1000, 5);
23     playTone(350, 1.5);
24     wait(2);
25     playTone(750, 3);
26     playTone(1800, 6);
27     wait(4);
28
29
30     /* =====
31     ===== STUDENT ALARM CODE (END HERE) =====
32     ===== */
33
34
```

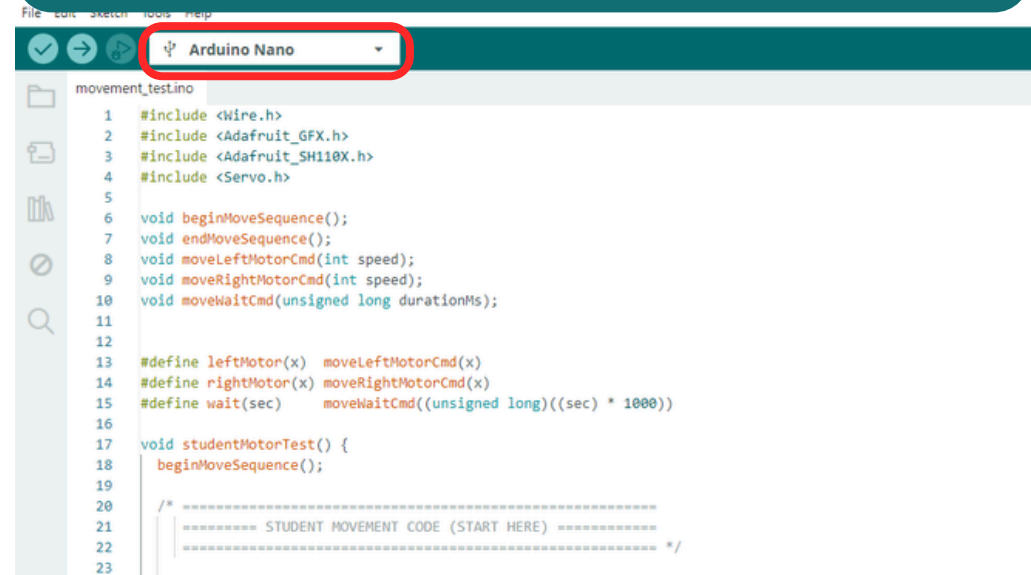
Get your code checked before uploading!

Once you are happy with your alarm tone, move onto programming your movement path

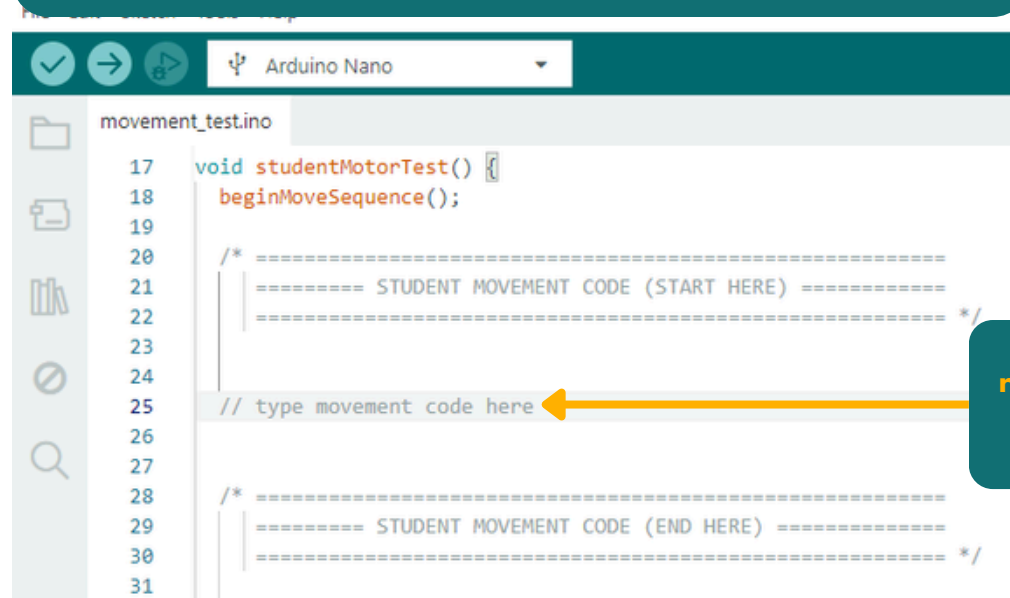
17 Go back to your browser (do not close alarm-code.ino) and open movement-code.ino



18 Repeat steps 8-11 to set up the board and USB port



19 Type your code inside the STUDENT MOVEMENT CODE section



Delete this line and replace with leftMotor, rightMotor and wait commands

`leftMotor(speed);`
`rightMotor(speed);`

Fast forward → 90
Moderate forward → 30
Slow forward → 10
Stop → 0
Slow reverse → -10
Moderate reverse → -30
Fast forward → -90

Do not exceed these speeds in the testing arena!

Examples:

- `leftMotor(30);`
`rightMotor(30);` → Bot will move forward at a moderate speed
- `leftMotor(-10);`
`rightMotor(-10);` → Bot will move backwards at a slow speed
- `rightMotor(90);`
`leftMotor(-90);` → Bot will rotate left at a fast speed
- `rightMotor(0);`
`leftMotor(0);` → Bot will stop

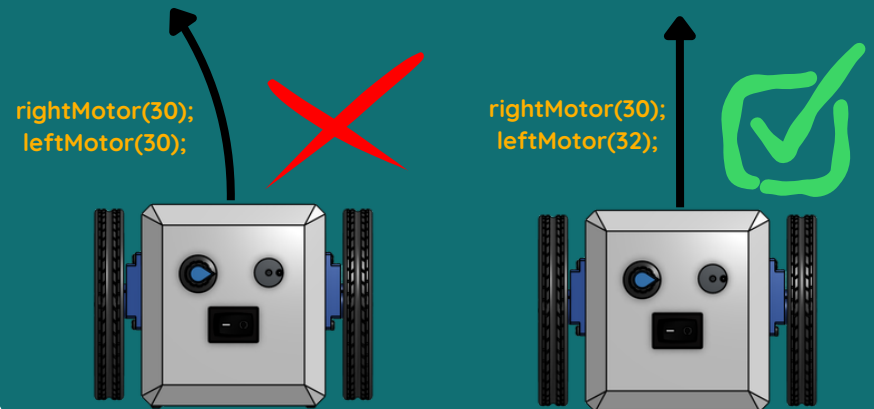
`wait(duration);`

How long the previous motor commands will run for

Examples:

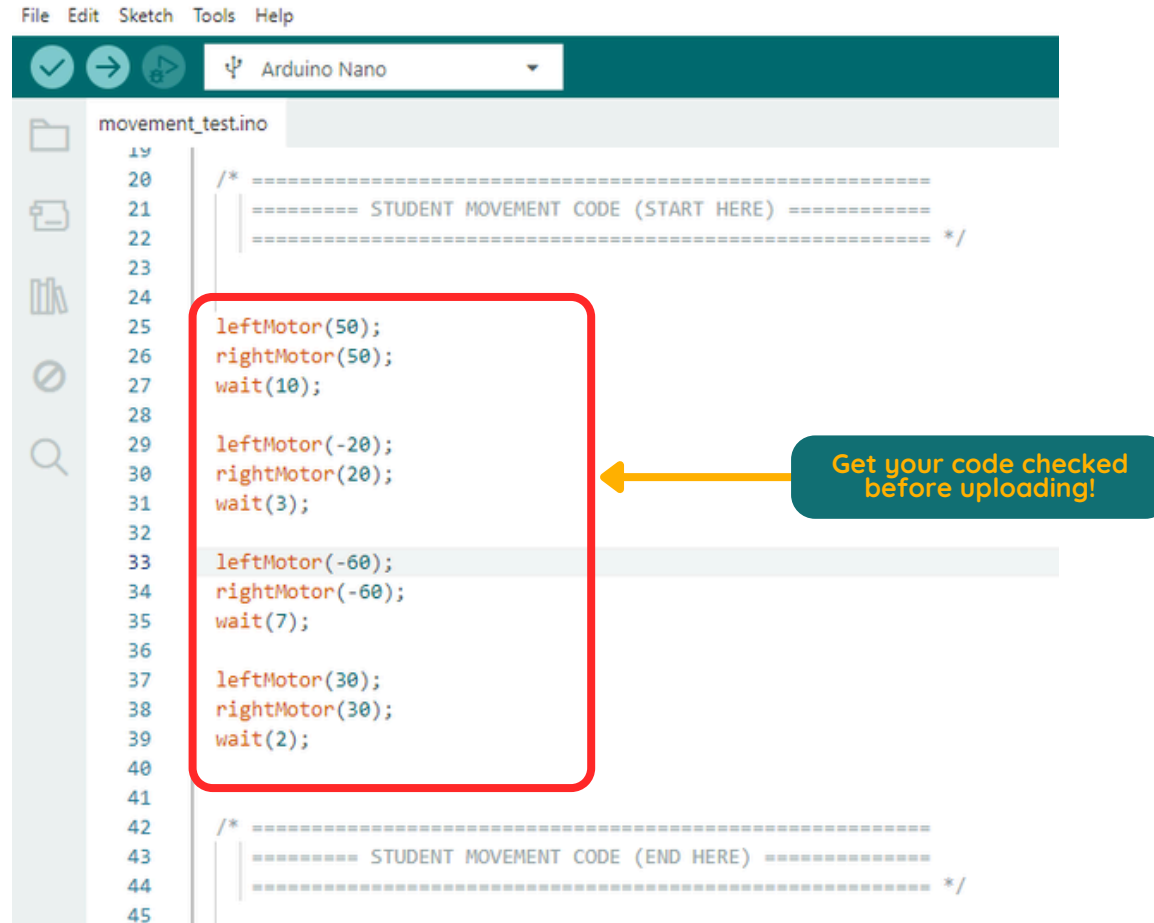
- `leftMotor(30);`
`rightMotor(30);`
`wait(2);`
→ Bot will move forwards for 2 seconds

The motors are not always precise so even though you may assign the same values to the left and right motors, the bot may slightly turn left or right instead of going straight. To counteract this, slightly raise or lower one of the motor's values until the bot drives straight.



20

Click on the **arrow** to upload your alarm code to the bot



```

File Edit Sketch Tools Help
[Checkmark] [Arrow] [Play] [USB] Arduino Nano

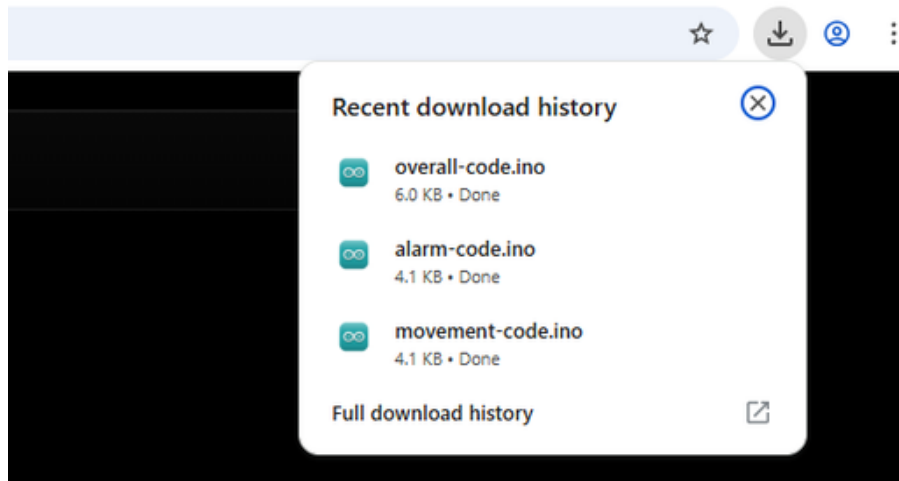
movement_test.ino
19
20  /* =====
21  |===== STUDENT MOVEMENT CODE (START HERE) =====
22  |===== */
23
24
25  leftMotor(50);
26  rightMotor(50);
27  wait(10);
28
29  leftMotor(-20);
30  rightMotor(20);
31  wait(3);
32
33  leftMotor(-60);
34  rightMotor(-60);
35  wait(7);
36
37  leftMotor(30);
38  rightMotor(30);
39  wait(2);
40
41
42  /* =====
43  |===== STUDENT MOVEMENT CODE (END HERE) =====
44  |===== */
45
  
```

Get your code checked before uploading!

Once you are happy with your movement path, you can move onto integrating your alarm and movement codes into the overall code.

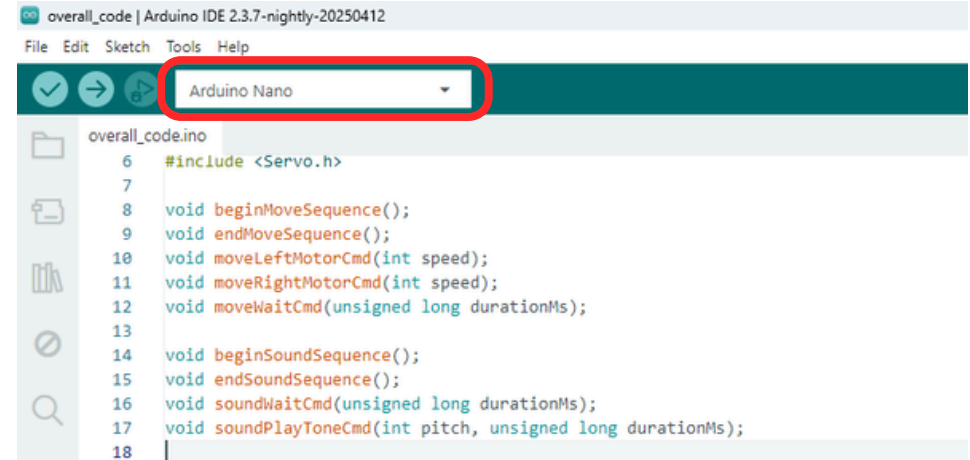
21

Go back to your browser (do not close movement-code.ino) and open overall-code.ino



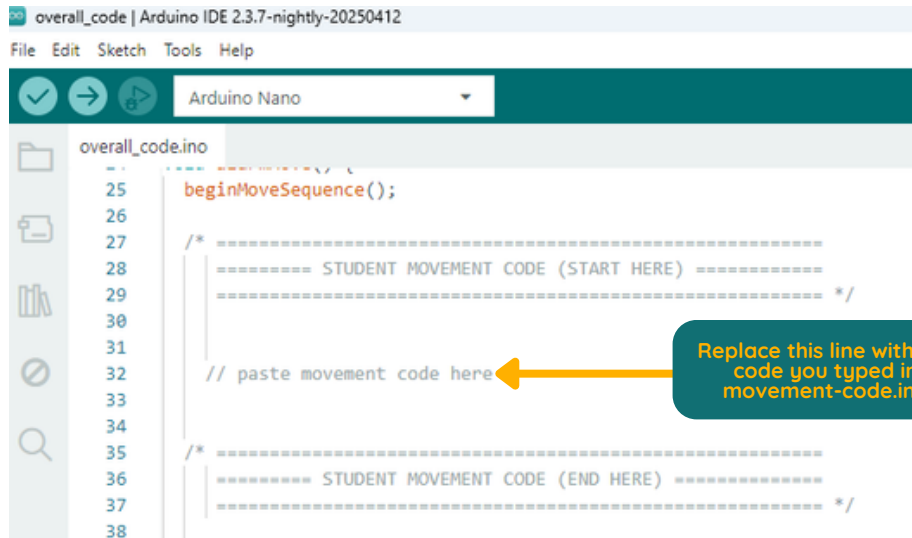
22

Repeat steps 8-11 to set up the board and USB port



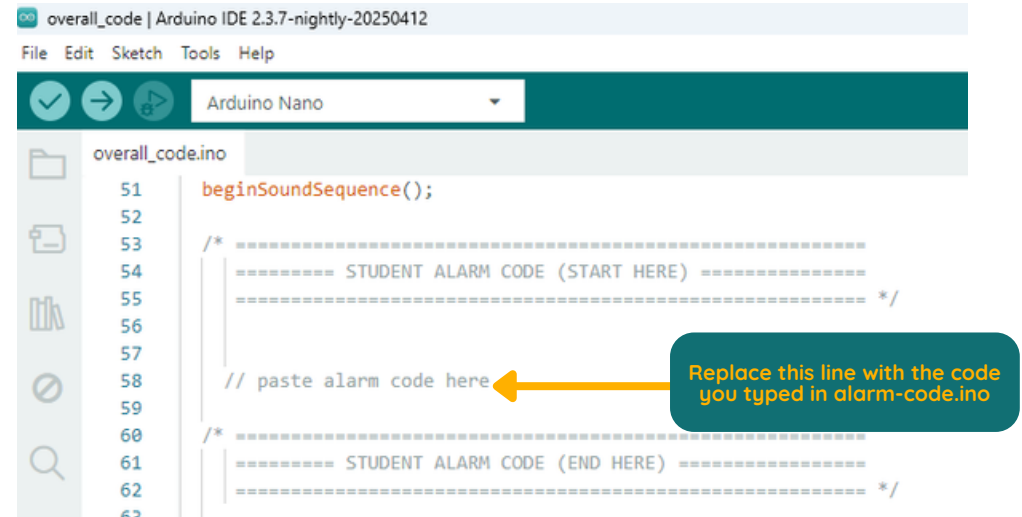
23

Copy and paste your typed code from movement-code.ino to overall_code.ino



24

Copy and paste your typed code from alarm_test.ino to overall_code.ino



25

Click on the arrow to upload your overall code to the bot



```

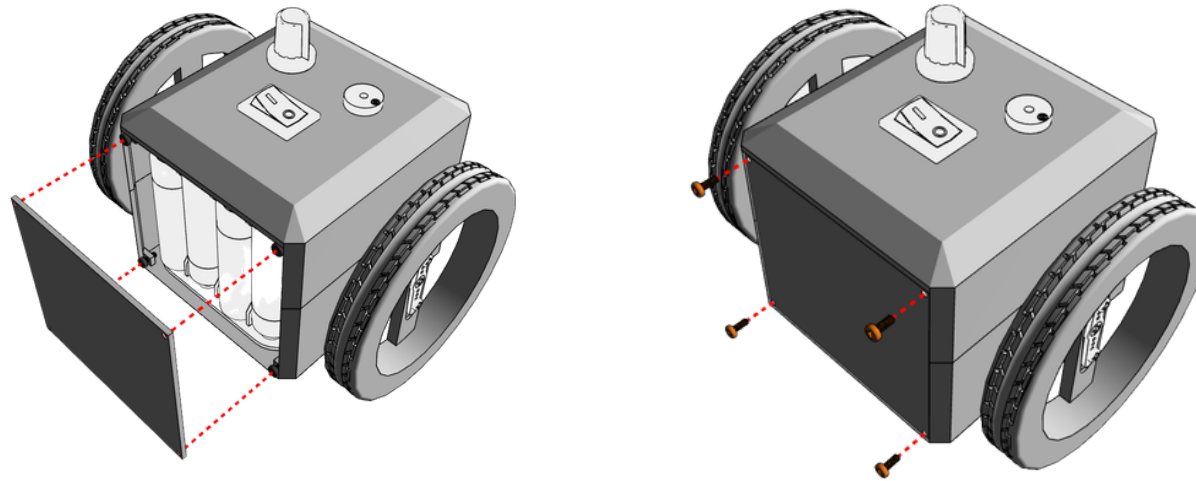
File Edit Sketch Tools Help
Arduino Nano
overall_code.ino
25 beginMoveSequence();
26
27 /*
28 ===== STUDENT MOVEMENT CODE (START HERE) =====
29 ===== */
30 leftMotor(50);
31 rightMotor(50);
32 wait(10);
33 leftMotor(-20);
34 rightMotor(20);
35 wait(3);
36 leftMotor(-60);
37 rightMotor(-60);
38 wait(7);
39 leftMotor(30);
40 rightMotor(30);
41 wait(2);
42
43 ===== STUDENT MOVEMENT CODE (END HERE) =====
44 ===== */
45 endMoveSequence();
46
47
48 #undef leftMotor
49 #undef rightMotor
50 #undef wait
51 #define playTone(p, sec) soundPlayToneCmd(p, (unsigned long)((sec) * 1000))
52 #define wait(sec) soundWaitCmd((unsigned long)((sec) * 1000))
53
54
55 void alarmSound() {
56 beginSoundSequence();
57
58 /*
59 ===== STUDENT ALARM CODE (START HERE) =====
60 ===== */
61 playTone(1000,5);
62 playTone(350, 1.5);
63 wait(2);
64 playTone(750,3);
65 playTone(1800,6);
66 wait(4);
67
68 ===== STUDENT ALARM CODE (END HERE) =====
69 ===== */
70

```

Be sure to email a copy of these codes to yourself so you can continue testing from home!

Congratulations you have finished building your Alarm Bot!
Please raise your hand to receive some final instructions.

ONCE YOU HAVE GIVEN THE INSTRUCTOR BACK THE BATTERIES, PLEASE SCREW ON THE BATTERY LID



HOW CAN I CONTINUE PROGRAMMING AT HOME?

The manual and code can be readily accessed from our website, projectbeacon.org.au, using the same password provided to you today. If you would like to continue working on the code you made during the workshop, be sure to email yourself a copy of the alarm and movement code before you leave. Should you run into any issues, be sure to send us an email at support@projectbeacon.org.au, and we will get back to you as soon as possible.

**THANKS FOR COMING BY!
STAY CURIOUS!**